Best Bike Paths (BBP)

Requirements Analysis & Design Presentation

Software Engineering II Project

Politecnico di Milano

A.Y. 2025-2026

1. System Goals

What BBP aims to achieve:

Primary Objectives

- **G02**: Enable cyclists to record personal trips (Manual/Automatic)
- **G06**: Provide human-in-the-loop validation of sensor data
- **G13**: Compute safety scores for bike paths
- **G15**: Detect road anomalies using mobile sensors
- **G16**: Merge multiple user contributions intelligently
- **G09**: Offer community access to safety information

Core Value: Transform individual cycling experiences into collective safety intelligence.

2. System Boundaries

World vs. Machine Phenomena:

World-Controlled (Real World)

- **WP01**: Users bike through streets
- **WP03**: Obstacles occur (potholes, debris)
- **WP05**: Meteorological conditions exist
- **WP04**: Mobile sensors detect movement

Machine-Controlled (BBP System)

- **SP06**: Infer anomalies from sensor data
- **SP10**: Display scored paths on map
- **SP12**: Merge path information from users
- **SP02**: Compute trip statistics

Boundary: BBP processes real-world phenomena but doesn't control physical streets.

3. Meaningful Use Case

Scenario: Emma's Daily Commute

1. **Recording:** Emma starts automatic recording on her smartphone
2. **Detection:** BBP identifies potholes via accelerometer spikes
3. **Validation:** Post-ride, Emma confirms/rejects each detection
4. **Publication:** She publishes the trip as "River Path - Morning"
5. **Merging:** System combines her data with other cyclists' reports
6. **Community Benefit:** Path appears in searches with score 8.5/10

Key Elements

- Sensor-based data acquisition
- Human validation step
- Community data merging
- Public safety benefit

4. Most Important Requirements

Critical Functional Requirements:

Data Acquisition

- **R05**: Record GPS coordinates at intervals
- **R06**: Generate TrackPoint sequences
- **R09**: Enrich with weather data
- **R33**: Handle sensor unavailability

Safety Intelligence

- **R15**: Link confirmed obstacles to streets
- **R24**: Update path scores based on reports
- **R31**: Retrieve external weather data
- **R16**: Respect publishability preferences

Why these matter: They enable reliable data collection and community safety intelligence.

5. Critical Assumptions & Constraints

Key Assumptions:

- **A01**: Users have smartphones with required sensors
- **A04**: Stable internet for data synchronization
- **A05**: Users provide truthful information
- **A07**: Device clocks are correctly set
- **A06**: External services provide accurate data

Design Constraints:

- **C01**: GDPR compliance for location data
- **C05**: Functionality with intermittent connectivity
- **C06**: Battery consumption optimization
- **C02**: Support for heterogeneous devices
- **C04**: Only validated data shown to community

Impact

These shape technical decisions: offline storage, privacy zones, sensor optimization.

6. Formal Analysis with Alloy

What we aim to prove:

Key Properties Verified

1. **Trip Coherence:** Track points are temporally ordered
2. **Obstacle Workflow:** Only CONFIRMED reports link to physical obstacles
3. **Majority Voting:** Path status determined by most frequent reports
4. **Data Consistency:** Manual trips have no track points, automatic trips do

Verification Results

- `confirmedMeansObstacle`: Verified - no counterexamples
- `bikePathStatusIsMajority`: Verified - majority rule holds
- `tripTimesAreConsistent`: Verified - temporal constraints satisfied

Value: Formal assurance of critical business logic correctness.

7. System Components

Microservices Architecture:

Client Layer

- Mobile Application
- UI Controller
- Sensor Manager
- Local DB Manager

Backend Services

- Trip Ingestion Service
- Validation Service
- Aggregation Service
- Scoring Service

Specialized Servers

- Trip Search Server
- Report Server
- Users Account Manager

Data Layer (Polyglot Persistence)

- User Data (PostgreSQL)
- Trips Data (Time-series DB)
- Score Data (PostGIS)
- Reports Data (PostgreSQL)

8. Component Interfaces

Key API Contracts:

Client-Facing (IBBPClientAPI)

- `POST /v1/trips/upload`
- `GET /v1/trips/search`
- `POST /v1/reports/obstacle`
- `PUT /v1/trips/id/publish`

Internal Services

- `ITripCompliance` (Validation)
- `ITripAggregation` (Merging)
- `IScoringCalculation` (Scoring)
- `IReportManagement` (Reports)

External Adapters

- `IMeteoProvider`: Weather API wrapper
- `IMapProvider`: OpenStreetMap integration

Design: Clear separation with DTOs for type safety.

9a. Architectural Styles & Patterns

Microservices Architecture

Why: Independent scalability, fault isolation

- Each service deployable independently
- Technology heterogeneity allowed
- Clear bounded contexts

API Gateway Pattern

Why: Single entry point, cross-cutting concerns

- Request routing
- Authentication/SSL termination
- Rate limiting

CQRS Pattern

Why: Optimize read vs. write performance

- Write side: Trip ingestion pipeline
- Read side: Dedicated search server
- Prevents resource contention

9b. Architectural Styles & Patterns

Containerization & Orchestration

Technology: Kubernetes (as per DD Chapter 2.3)

- Containerized microservices
- Automated scaling during peak hours
- Self-healing with health checks
- Service discovery & load balancing

Polyglot Persistence

Why: Right tool for each data type

- Time-series for sensor logs
- PostGIS for geospatial queries
- Relational for user data

Deployment Infrastructure

- Kubernetes cluster with Ingress Controller
- Secrets & ConfigMaps for secure configuration
- Cloud infrastructure (AWS RDS, Timestream)

10. Meaningful Interactions

Trip Processing Pipeline:

1. **Recording:** Mobile App → Sensor Manager → Local DB
2. **Upload:** Network Manager → API Gateway → Trip Ingestion
3. **Validation:** Ingestion → Validation Service → Map Matching
4. **Aggregation:** Validation → Aggregation Service → Voting Engine
5. **Scoring:** Aggregation → Scoring Service → Score update
6. **Search:** User → Search Server → Spatial queries → Results

Key Interaction Patterns

- **Asynchronous Processing:** HTTP 202 responses for background jobs
- **Eventual Consistency:** Scores updated after processing
- **Human-in-the-loop:** Validation before obstacle confirmation
- **Offline Resilience:** Local buffering during connectivity loss

11a. User Interface Design

Mobile-First Approach for Cyclists

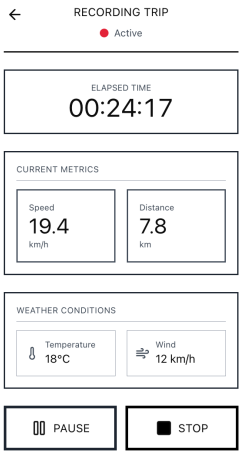
Auto Recording Interface

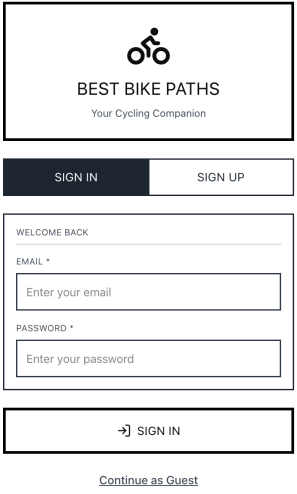
- Real-time metrics display
- Weather conditions integration
- Simple start/stop controls
- Designed for safe cycling interaction

Trip Search Interface

- Color-coded paths by safety score
- Filtering by score/conditions
- Obstacle visualization on map
- Accessible to unregistered users

11b. User Interface Design







12. Implementation & Testing Strategy

Bottom-Up Approach:

1. **Step 0:** Data Access Layer & External Adapters
2. **Step 1:** Core Authentication & User Management
3. **Step 2:** Core Algorithms (Scoring, Detection)
4. **Step 3:** Service Integration (Ingestion, Reports)
5. **Step 4:** Search & Visualization Layer
6. **Step 5:** Client Application Integration

Testing Strategy

- **Unit:** Test drivers for sensor modules
- **Integration:** Service-to-service communication
- **Performance:** 50k concurrent users simulation
- **Security:** GDPR compliance verification
- **Failure:** Offline/crash recovery testing
- **UAT:** Real cyclists with realistic tasks

Mapping to Components: Each step builds on tested lower-level modules.

13. Requirements Traceability

Mapping RASD Requirements to DD Components:

Requirement	Implementing Component(s)
R01, R02 (Registration/Login)	Users Account Manager Service
R04, R05, R06 (Trip Recording)	Trip Manager (Mobile) + Trip Ingestion Service
R11, R12, R14 (Obstacle Reporting)	Report Server (Ingestion + Validation)
R15, R18 (Obstacle-Street Association)	Report Validation Service + MapAPI Wrapper
R24 (Score Updates)	Trip Scoring Service
R31 (Weather Data)	WeatherAPI Wrapper
R25, R26 (Path Browsing)	Trip Search Server + Map Manager
R33, R34 (Sensor Handling)	Sensor Manager (Mobile)

14. Verification

Verification

- All 35 requirements mapped to architectural components
- Critical requirements have dedicated services
- Non-functional requirements addressed through patterns

Conclusion

Thank you for your attention!